General

Template

```cpp
#pragma GCC optimize("O3", "unroll-loops")
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
#define endl '\n'
#define all(v) (v).begin(), (v).end()

void solve(){

}

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    ll t = 1; cin >> t;
    while(t--) solve();
}
```

Grafos

BellmanFord

```cpp
struct BellmanFord {
  struct Edge {
    int from, to, weight;
  };
  int n, last_updated = -1;
  const ll INF = 1e18;
  vector<int> p, dist;
  BellmanFord(vector<Edge>& G, int s, int nn) {
    n = nn;
    dist.assign(n + 2, INF);
    p.assign(n + 2, -1);
    dist[s] = 0;
    for (int i = 1; i <= n; i++) {
      last_updated = -1;
      for (Edge& e : G)
        if (dist[e.from] + e.weight < dist[e.to]) {
          dist[e.to] = dist[e.from] + e.weight;
          p[e.to] = e.from;
          last_updated = e.to;
        }
    }
  }
  bool getCycle(vector<int>& cycle) {
    if (last_updated == -1) return false;
    for (int i = 0; i < n - 1; i++) last_updated = p[last_updated];
    for (int x = last_updated;; x = p[x]) {
      cycle.push_back(x);
      if (x == last_updated and cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return true;
  }
};
```

BinaryLifting

```cpp
void BinaryLifting(ll node) {
  stack<pair<ll, ll>> st;
  st.push({node, -1});
  while (st.size()) {
    auto [n, p] = st.top();
    st.pop();
    for (auto v : tree[n]) {
      if (p == v) continue;
      st.push({v, n});
      bl[v][0] = n;
      for (ll j = 1; j < 33; j++) {
        if (bl[v][j - 1] != -1) {
          bl[v][j] = bl[bl[v][j - 1]][j - 1];
        } else {
          bl[v][j] = -1;
        }
      }
    }
  }
}
```

Djikstra

```cpp
vector<ll> dist;
vector < vector<pair<ll, ll>> g;

void djikstra() {
  priority_queue<pair<ll, ll>, vector<pair<ll, ll>>, greater<pair<ll, ll>>> pq;
  pq.push({0, 0});  // nodo_inicio, distancia
  while (q.size()) {
    auto [c, node] = pq.top();
    pq.pop();
    if (dist[node] > c) {
      dist[node] = c;
      for (auto [c_, v_] : g[node]) {
        if (dist[v_] > c + c_) pq.push({c + c_, v_});
      }
    }
  }
}
```

Floyd-Warshall

```cpp
for (int i = 1; i <= n; i++) {
  for (int j = 1; j <= n; j++) {
    if (i == j)
      distance[i][j] = 0;
    else if (adj[i][j])
      distance[i][j] = adj[i][j];
    else
      distance[i][j] = INF;
  }
}
for (int k = 1; k <= n; k++) {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      distance[i][j] = min(distance[i][j], distance[i][k] + distance[k][i]);
    }
  }
}
```

Kruskal

```cpp
struct Edge {
  int a;
  int b;
  ll w;
  Edge(int a_, int b_, ll w_) : a(a_), b(b_), w(w_) {}
};
bool c_edge(Edge& a, Edge& b) { return a.w < b.w; }
ll Kruskal() {
  int n = G.size();
  union_find sets(n);
  vector<Edge> edges;
  for (int i = 0; i < n; i++) {
    for (auto eg : G[i]) {
      // node i to node eg.first with cost eg.second
      edges.emplace_back(i, eg.first, eg.second);
    }
  }
  sort(edges.begin(), edges.end(), c_edge);
  ll min_cost = 0;
  for (Edge e : edges) {
    if (sets.sameSet(e.a, e.b) != true) {
      tree.emplace_back(e.a, e.b, e.w);
      min_cost += e.w;
      sets.unionSet(e.a, e.b);
    }
  }
  return min_cost;
}
```

Lca

```cpp
struct LCA {
  vector<vector<ll>> T, parent;
  vector<ll> depth, vis, comp;
  ll LOGN, V;
  LCA(vector<vector<ll>>& T) : T(T) {
    V = (ll)T.size();
    LOGN = 1;
    while ((1LL << LOGN) <= max(1LL, V)) LOGN++;
    parent.assign(V, vector<ll>(LOGN, 0));
    depth.assign(V, 0);
    vis.assign(V, 0);
    comp.assign(V, -1);
    ll id = 0;
    for (ll u = 0; u < V; u++) {
      if (!vis[u]) {
        dfs(u, u, id++);
      }
    }
  }

  void dfs(ll u, ll p, ll id) {
    vis[u] = 1;
    comp[u] = id;
    parent[u][0] = p;
    for (ll j = 1; j < LOGN; j++)
      parent[u][j] = parent[parent[u][j - 1]][j - 1];

    for (ll v : T[u]) {
      if (!vis[v]) {
        depth[v] = depth[u] + 1;
        dfs(v, u, id);
      }
    }
  }
  ll query(ll u, ll v) {
    if (comp[u] != comp[v]) return -1;
    if (depth[u] < depth[v]) swap(u, v);
    ll k = depth[u] - depth[v];
    for (ll j = LOGN - 1; j >= 0; j--) {
      if ((k >> j) & 1) u = parent[u][j];
    }
    if (u == v) return u;
    for (ll j = LOGN - 1; j >= 0; j--) {
      if (parent[u][j] != parent[v][j]) {
        u = parent[u][j];
        v = parent[v][j];
      }
    }
    return parent[u][0];
  }
};
```

Flujo Y Conectividad

Estructuras De Datos

Fenwick

```cpp
struct fenwick_tree {
  vector<ll> bit;
  int n;
  fenwick_tree(int n) : n(n) { bit.assign(n, 0); }
  fenwick_tree(vector<ll>& a) : fenwick_tree(a.size()) {
    for (size_t i = 0; i < a.size(); i++) {
      bit[i] += a[i];
      if ((i | (i + 1)) < a.size()) bit[i | (i + 1)] += bit[i];
    }
  }
  ll sum(int r) {
    ll ret = 0;
    for (; r >= 0; r = (r & (r + 1)) - 1) ret += bit[r];
    return ret;
  }
  ll sum(int l, int r) { return sum(r) - sum(l - 1); }
  void add(int idx, ll delta) {
    for (; idx < n; idx = idx | (idx + 1)) bit[idx] += delta;
  }
};
```

Fenwick 2D

```cpp
struct fenwick_tree_2d {
  int N, M;
  vector<vector<ll>> BIT;
  fenwick_tree_2d(int N, int M) : N(N), M(M) {
    BIT.assign(N + 1, vector<ll>(M + 1, 0));
  }
  void update(int x, int y, ll v) {
    for (int i = x; i <= N; i += (i & -i))
      for (int j = y; j <= M; j += (j & -j)) BIT[i][j] += v;
  }
  ll sum(int x, int y) {
    ll s = 0;
    for (int i = x; i > 0; i -= (i & -i))
      for (int j = y; j > 0; j -= (j & -j)) s += BIT[i][j];
    return s;
  }
  ll query(int x1, int y1, int x2, int y2) {
    return sum(x2, y2) - sum(x2, y1 - 1) - sum(x1 - 1, y2) +
           sum(x1 - 1, y1 - 1);
  }
};
```

Fenwick Walk

```cpp
struct fenwick_tree_with_walk : fenwick_tree {
  ll walk(ll k) {
    if (k <= 0 || k > sum(n - 1)) return n;
    ll i = -1, s = 1;
    while ((s << 1) < n) s <<= 1;
    while (s > 0) {
      ll m = i + s;
      if (m < n && bit[m] < k) {
        i = m;
        k -= bit[m];
      }
      s >>= 1;
    }
    return i + 1;
  }
};
```

Line Container

```cpp
const ll INF = 1e15;
struct Line {
  mutable ll a, b, c;
  bool operator<(Line r) const {
    return a < r.a;
  }
  bool operator<(ll x) const {
    return c < x;
  }
};
// dynamically insert `a*x + b` lines and query for maximum
// at any x all operations have complexity O(log N)
struct LineContainer: multiset < Line, less < >> {
  ll div(ll a, ll b) {
    return a / b - ((a ^ b) < 0 && a % b);
  }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x -> c = INF, 0;
    if (x -> a == y -> a) x -> c = x -> b > y -> b ? INF : -INF;
    else x -> c = div(y -> b - x -> b, x -> a - y -> a);
    return x -> c >= y -> c;
  }

  void add(ll a, ll b) {
    // a *= -1, b *= -1 // for min
    auto z = insert({
      a,
      b,
      0
    }), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x) -> c >= y -> c) isect(x, erase(y));
  }

  ll query(ll x) {
    if (empty()) return -INF; // INF for min
    auto l = * lower_bound(x);
    return l.a * x + l.b;
    // return -l.a * x - l.b; // for min
  }
};
```

Mo

```cpp
struct query {
  ll l, r, i;
};
template<class T, typename U, class T2>
struct mo_algorithm {
  vector<U> ans;
  template<typename... Args>
  mo_algorithm(vector<T> &v, vector<query> &queries, Args... args) {
    T2 ds(args...);
    ll block_sz = sqrtl(v.size());
    ans.assign(queries.size(), -1);
    sort(queries.begin(), queries.end(), [&](auto &a, auto &b) {
      return a.l/block_sz != b.l/block_sz ? a.l/block_sz < b.l/block_sz : a.r < b.r;
    });
    ll l = 0, r = -1;
    for (query q : queries) {
      while (l > q.l) ds.add(v[--l]);
      while (r < q.r) ds.add(v[++r]);
      while (l < q.l) ds.remove(v[l++]);
      while (r > q.r) ds.remove(v[r--]);
      ans[q.i] = ds.answer(q);
    }
  }
};
```

Segment Tree

```cpp
template<class T, T m_(T, T)> struct segment_tree{
  int n; vector<T> ST;
  segment_tree(){}
  segment_tree(vector<T> &a){
    n = a.size(); ST.resize(n << 1);
    for (int i=n;i<(n<<1);i++)ST[i]=a[i-n];
    for (int i=n-1;i>0;i--)ST[i]=m_(ST[i<<1],ST[i<<1|1]);
  }
  void update(int pos, T val){ // replace with val
    ST[pos += n] = val;
    for (pos >>= 1; pos > 0; pos >>= 1)
      ST[pos] = m_(ST[pos<<1], ST[pos<<1|1]);
  }
  T query(int l, int r){ // [l, r]
    T ansL, ansR; bool hasL = 0, hasR = 0;
    for (l += n, r += n + 1; l < r; l >>= 1, r >>= 1) {
      if (l & 1)
        ansL=(hasL?m_(ansL,ST[l++]):ST[l++]),hasL=1;
      if (r & 1)
        ansR=(hasR?m_(ST[--r],ansR):ST[--r]),hasR=1;
    }
    if (!hasL) return ansR; if (!hasR) return ansL;
    return m_(ansL, ansR);
  }
};
```

Segment Tree Lazy

```cpp
template<
  class T1, // answer value stored on nodes
  class T2, // lazy update value stored on nodes
  T1 merge(T1, T1),
  void pushUpd(T2& /*parent*/, T2& /*child*/, int, int, int, int), // push update value from a node to another. parent -> child
  void applyUpd(T2&, T1&, int, int)           // apply the update value of a node to its answer value. upd -> ans
>
struct segment_tree_lazy{
  vector<T1> ST; vector<T2> lazy; vector<bool> upd;
  int n;
  void build(int i, int l, int r, vector<T1>&values){
    if (l == r){
      ST[i] = values[l];
      return;
    }
    build(i << 1, l, (l + r) >> 1, values);
    build(i << 1 | 1, (l + r) / 2 + 1, r, values);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  segment_tree_lazy() {}
  segment_tree_lazy(vector<T1>&values){
    n = values.size(); ST.resize(n << 2 | 3);
    lazy.resize(n << 2 | 3); upd.resize(n << 2 | 3, false);
    build(1, 0, n - 1, values);
  }
  void push(int i, int l, int r){
    if (upd[i]){
      applyUpd(lazy[i], ST[i], l, r);
      if (l != r){
        pushUpd(lazy[i], lazy[i << 1], l, r, l, (l + r) / 2);
        pushUpd(lazy[i], lazy[i << 1 | 1], l, r, (l + r) / 2 + 1, r);
        upd[i << 1] = 1;
        upd[i << 1 | 1] = 1;
      }
      upd[i] = false;
      lazy[i] = T2();
    }
  }
  void update(int i, int l, int r, int a, int b, T2 &u){
    if (l >= a and r <= b){
      pushUpd(u, lazy[i], a, b, l, r);
      upd[i] = true;
    }
    push(i, l, r);
    if (l > b or r < a) return;
    if (l >= a and r <= b) return;
    update(i << 1, l, (l + r) >> 1, a, b, u);
    update(i << 1 | 1, (l + r) / 2 + 1, r, a, b, u);
    ST[i] = merge(ST[i << 1], ST[i << 1 | 1]);
  }
  void update(int a, int b, T2 u){
    if (a > b){
      update(0, b, u);
      update(a, n - 1, u);
      return ;
    }
    update(1, 0, n - 1, a, b, u);
  }
  T1 query(int i, int l, int r, int a, int b){
    push(i, l, r);
    if (a <= l and r <= b)
      return ST[i];
    int mid = (l + r) >> 1;
    if (mid < a)
      return query(i << 1 | 1, mid + 1, r, a, b);
    if (mid >= b)
      return query(i << 1, l, mid, a, b);
    return merge(query(i << 1, l, mid, a, b), query(i << 1 | 1, mid + 1, r, a, b));
  }
  T1 query(int a, int b){
    if (a > b) return merge(query(a, n - 1), query(0, b));
    return query(1, 0, n - 1, a, b);
  }
};
```

Segment Tree Walk

```cpp
template<class T, T m_(T, T)> struct segment_tree_with_walk
 : segment_tree<T,m_> {
  int walk(int l, T k) {
    vector<int> L, R;
    int r = this->n - 1;
    for (l += this->n, r += this->n + 1; l < r; l >>= 1, r
>>= 1) {
      if (l & 1) L.push_back(l++);
      if (r & 1) R.push_back(--r);
    }
    reverse(R.begin(), R.end());
    L.insert(L.end(), R.begin(), R.end());
    for (int v : L) {
      if (this->ST[v] >= k) {
        while (v < this->n) v = (this->ST[v << 1] >= k) ? (
v << 1) : (v << 1 | 1);
        return v - this->n;
      }
    }
    return -1;
  }
};
```

Sparse Table

```cpp
template <typename T, T m_(T, T)>
struct sparse_table {
  int n;
  vector<vector<T>> table;
  sparse_table() {}
  sparse_table(vector<T> &arr) {
    n = arr.size();
    int k = log2_floor(n) + 1;
    table.assign(n, vector<T>(k));
    for (int i = 0; i < n; i++)
      table[i][0] = arr[i];
    for (int j = 1; j < k; j++)
      for (int i = 0; i + (1 << j) <= n; i++)
        table[i][j] = m_(table[i][j-1], table[i+(1<<(j-1))]
[j-1]);
  }
  T query(int l, int r) {
    int k = log2_floor(r - l + 1);
    return m_(table[l][k], table[r-(1 << k)+1][k]);
  }
  int log2_floor(int n) { return n ? __builtin_clzll(1) - _
_builtin_clzll(n) : -1; }
};
```

Sqrt Decomposition

```cpp
template <class T, T merge(T,T)>
struct sqrt_decomp {
  int n,sq;
  vector<T> st,bl;
  sqrt_decomp(vector<T> & a){
    n = a.size(); st.resize(n);
    sq = sqrtl(n); bl.resize((n+sq-1)/sq);
    for (int i = 0; i < n; i++){
      st[i] = a[i];
      bl[i/sq] = ((i%sq == 0)?st[i]:merge(bl[i/sq],st[i]));
    }
  }
  void update(int pos, T val) {
    st[pos] = val;
    int block = pos/sq, lb = block*sq, rb = min(block*sq+sq
-1,n-1);
    T vl;
    for (int i = lb; i <= rb; i++)
      vl = ((i == lb)?st[i]:merge(vl,st[i]));
    bl[block] = vl;
  }
  T query(int l, int r){
    T res; bool hasR = 0;
    for (int i = l/sq; i <= r/sq; i++){
      int lb = i*sq, rb = i*sq+sq-1;
      if (l <= lb && rb <= r){
        res = (hasR?merge(res,bl[i]):bl[i]), hasR = 1;
        continue;
      }
      for (int j = max(l,lb); j <= min(rb,r); j++)
        res = (hasR?merge(res,st[j]):st[j]), hasR = 1;
    }
    return res;
  }
};
```

Union Find

```cpp
struct union_find {
  vector<int> e;
  union_find(int n) { e.assign(n, -1); }
  int findSet(int x) { return (e[x] < 0 ? x : e[x] = findSe
t(e[x])); }
  bool sameSet(int x, int y) { return findSet(x) == findSet
(y); }
  int size(int x) { return -e[findSet(x)]; }
  bool unionSet(int x, int y) {
    x = findSet(x), y = findSet(y);
    if (x == y) return 0;
    if (e[x] > e[y]) swap(x, y);
    e[x] += e[y], e[y] = x;
    return 1;
  }
};
```

Matematica

Strings

Programacion Dinamica

Miscelaneo